

# System Design: Rate Limiter

Distributed API Rate Limiting — Senior Engineer Interview Walkthrough

## 1. Requirements & Capacity

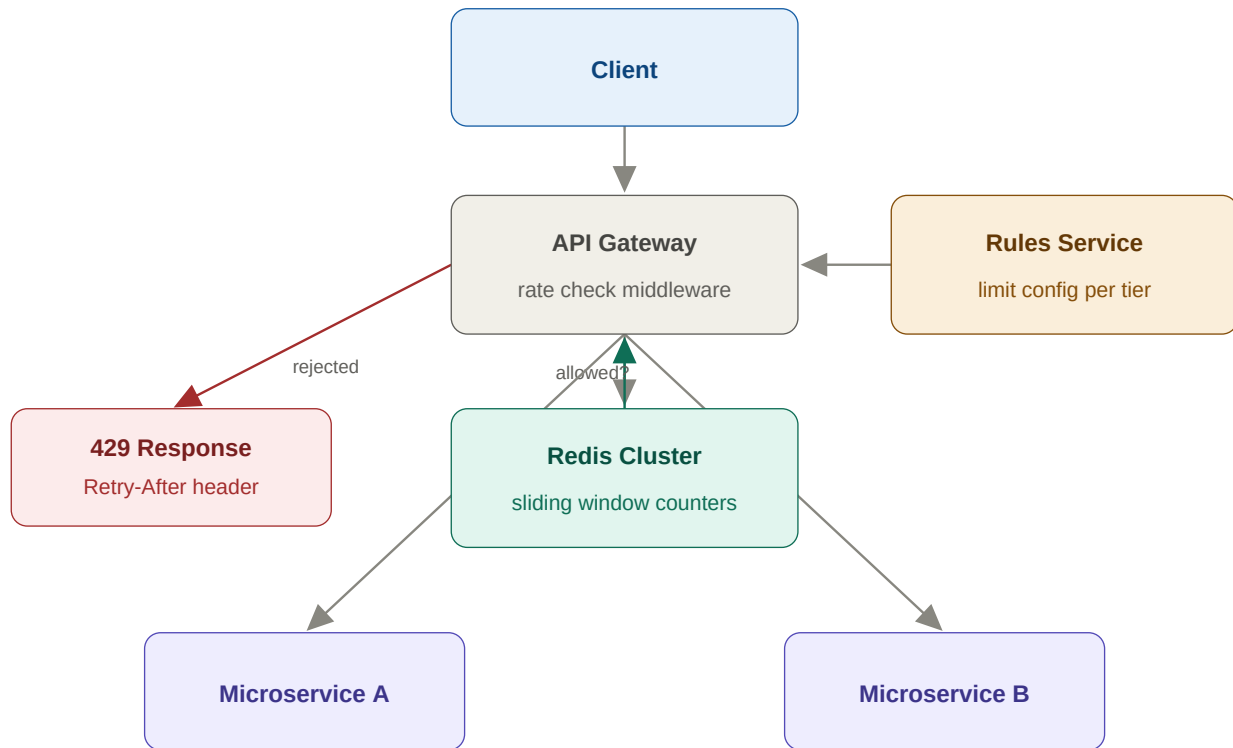
Think out loud: "Per user\_id or API key? Hard reject (429) or throttle? Sliding window? Distributed across 50+ servers? Yes to all. Check must add < 5ms."

- Limit N requests per time window per user/API key
- Return HTTP 429 + Retry-After header when exceeded
- Different limits per endpoint or pricing tier
- Distributed — consistent decision across all API servers

| Scale | Users/RPS     | Architecture                                      | Storage         |
|-------|---------------|---------------------------------------------------|-----------------|
| Low   | Single server | In-memory counter + hash map, reset via cron      | No Redis needed |
| Mid   | 10 servers    | Redis single node, Lua script for atomicity       | ~1 GB Redis     |
| High  | 50+ servers   | Redis cluster, sliding window counter, Lua atomic | ~10 GB Redis    |

## 2. Architecture Diagram

Think out loud: "Rate limiter lives in the API Gateway — it stops excess load before it reaches any service. The check calls Redis atomically via Lua script."



Architecture: every request → API Gateway → Redis Lua check → allow (pass to service) or reject (429). Rules Service provides limit config per tier.

### 3. Algorithm: Sliding Window Counter (recommended)

Think out loud: "I'll compare 4 algorithms. The interviewer wants to see I know the trade-offs, not just name-drop token bucket."

- Fixed Window Counter: simple, but boundary burst allows 2x rate in 2 seconds spanning window boundary
- Sliding Window Log: accurate, but stores every timestamp → too much memory at scale
- Token Bucket: great for burst tolerance (common in public APIs like GitHub), 2 Redis fields per user
- Sliding Window Counter (OUR CHOICE): hybrid. 2 counters, weighted estimate. ~0.003% error. Used by Cloudflare + Stripe

**Why this choice:** Sliding Window Counter:  $\text{rate} \approx \text{prev\_count} \times (1 - \text{elapsed}/\text{window}) + \text{curr\_count}$ . Only 2 Redis keys per user. Executed as atomic Lua script — prevents TOCTOU race. Memory-efficient at 10M+ users.

#### Redis Lua Script (atomic)

```

local curr = tonumber(redis.call("GET", KEYS[1]) or 0) local prev =
tonumber(redis.call("GET", KEYS[2]) or 0) local elapsed = tonumber(ARGV[3]) %
tonumber(ARGV[2]) local rate = prev * (1 - elapsed/tonumber(ARGV[2])) + curr if rate >=
tonumber(ARGV[1]) then return 0 end redis.call("INCR", KEYS[1]) redis.call("EXPIRE", KEYS[1],
tonumber(ARGV[2])*2) return 1
  
```

4. DB Schema & API

| rate_limit_rules |                  |                           |
|------------------|------------------|---------------------------|
| PK               | rule_id          | UUID                      |
|                  | identifier_type  | ENUM (USER/API_KEY/IP)    |
|                  | endpoint_pattern | VARCHAR (/api/v1/posts/*) |
|                  | plan_tier        | VARCHAR (FREE/PRO/ENT)    |
|                  | limit_count      | INT                       |
|                  | window_seconds   | INT                       |

|                                                                                                                          |                                     |                                   |
|--------------------------------------------------------------------------------------------------------------------------|-------------------------------------|-----------------------------------|
| Schema: rule-table in Postgres connected to Redis (TTL: 60s). Gateway reads rules once per deployment, caches in memory. |                                     |                                   |
| GET                                                                                                                      | /api/v1/rate-limit/status/{user_id} | Current usage + remaining quota   |
| POST                                                                                                                     | /api/v1/rate-limit/rules            | Create new rule for tier/endpoint |
| DELETE                                                                                                                   | /api/v1/rate-limit/rules/{rule_id}  | Delete a rule                     |

Response headers on every call: X-RateLimit-Limit: 100 | X-RateLimit-Remaining: 42 | X-RateLimit-Reset: 1700000120 | Retry-After: 58 (on 429)

| Decision    | Choice                 | Gain                 | Trade-off                  |
|-------------|------------------------|----------------------|----------------------------|
| Algorithm   | Sliding window counter | Low memory, ~0 error | 0.003% boundary inaccuracy |
| Storage     | Redis cluster          | Sub-ms, atomic Lua   | Redis HA needed            |
| Placement   | API Gateway            | Stops load early     | Vendor coupling            |
| Consistency | Eventual (multi-node)  | Low latency          | Slight over-serving        |